Code snippets

Kopieer één volledig codeblok naar een nieuw Python-bestand of een lege notebookcel en voer het uit. Elk voorbeeld bevat eigen imports en voorbeelddata; je hoeft eerdere blokken niet uit te voeren. Vervang daarna de data en eenheden door die van jouw experiment.
Installeer zo nodig de packages in je actieve practicum-environment, via de terminal:

Terminal / uitvoer

```
conda install numpy matplotlib scipy pandas
```

Zie Python-tips voor uitleg en Python installeren [1] voor de werkomgeving.

Meetpunten plotten en een figuur opslaan

Dit voorbeeld slaat `meetpunten.png` op in de huidige werkmap. Een bestaand bestand met die naam wordt vervangen. Sla de figuur op vóór `plt.show()`.

Python

```python
import numpy as np
import matplotlib.pyplot as plt

tijd = np.array([0, 1, 2, 3, 4])
spanning = np.array([0.1, 1.2, 1.9, 3.1, 4.0])

fig, ax = plt.subplots()
ax.plot(tijd, spanning, "o")
ax.set_xlabel("Tijd (s)")
ax.set_ylabel("Spanning (V)")
fig.tight_layout()
fig.savefig("meetpunten.png", dpi=300)
plt.show()
```

Meetpunten met foutbalken

De foutbalken zijn hier standaardonzekerheden (één sigma). `xerr` kan op dezelfde manier worden toegevoegd voor horizontale foutbalken.

Python

```python
import numpy as np
import matplotlib.pyplot as plt

tijd = np.array([0, 1, 2, 3, 4])
positie = np.array([0.1, 1.2, 1.9, 3.1, 4.0])
sigma = np.array([0.1, 0.2, 0.1, 0.2, 0.1])

fig, ax = plt.subplots()
ax.errorbar(tijd, positie, yerr=sigma, fmt="o", capsize=3)
ax.set_xlabel("Tijd (s)")
ax.set_ylabel("Positie (m)")
fig.tight_layout()
plt.show()
```

Een rechte lijn fitten

Het model is `positie = a * tijd + b`. We nemen onafhankelijke meetfouten en verwaarloosbare onzekerheid in de tijd aan. `sigma` bevat positieve standaardonzekerheden in de positie. `absolute_sigma=True` gebruikt deze schaal ook voor de parameteronzekerheden.

Python

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

def lineair(t, a, b):
    return a * t + b

tijd = np.array([1, 2, 3, 4, 5], dtype=float)
positie = np.array([1.1, 1.9, 3.2, 3.9, 5.1])
sigma = np.full(5, 0.1)

popt, pcov = curve_fit(lineair, tijd, positie,
                       sigma=sigma, absolute_sigma=True)
perr = np.sqrt(np.diag(pcov))
print(f"a = {popt[0]:.3f} +/- {perr[0]:.3f} m/s")
print(f"b = {popt[1]:.3f} +/- {perr[1]:.3f} m")

t = np.linspace(tijd.min(), tijd.max(), 200)
fig, ax = plt.subplots()
ax.errorbar(tijd, positie, yerr=sigma, fmt="o", label="Meetdata")
ax.plot(t, lineair(t, *popt), label="Lineaire fit")
ax.set_xlabel("Tijd (s)")
ax.set_ylabel("Positie (m)")
ax.legend()
fig.tight_layout()
plt.show()
```

Je vindt ongeveer `a = 1.000 +/- 0.032 m/s` en `b = 0.040 +/- 0.105 m`. De onzekerheden zijn standaardonzekerheden, geen gegarandeerde grenzen. `curve_fit` neemt onzekerheden in de x-waarden niet mee; alleen `xerr` tekenen verandert de fit niet.

Een exponentiële afname fitten

Dit model beschrijft een spanning die naar nul afneemt: `U(t) = U0 * exp(-t / tau)`. De tijdconstante `tau` is positief. De startwaarden `p0` helpen bij deze niet-lineaire fit.

Python

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

def afname(t, U0, tau):
    return U0 * np.exp(-t / tau)

tijd = np.array([0, 1, 2, 3, 4, 5], dtype=float)
spanning = np.array([5.02, 3.01, 1.86, 1.10, 0.69, 0.40])
sigma = np.full(6, 0.05)

popt, pcov = curve_fit(afname, tijd, spanning, p0=[5, 2],
                       bounds=([0, 0.01], [np.inf, np.inf]),
                       sigma=sigma, absolute_sigma=True)
perr = np.sqrt(np.diag(pcov))
print(f"U0 = {popt[0]:.3f} +/- {perr[0]:.3f} V")
print(f"tau = {popt[1]:.3f} +/- {perr[1]:.3f} s")

t = np.linspace(0, 5, 200)
fig, ax = plt.subplots()
ax.errorbar(tijd, spanning, yerr=sigma, fmt="o", label="Meetdata")
ax.plot(t, afname(t, *popt), label="Exponentiële fit")
ax.set_xlabel("Tijd (s)")
ax.set_ylabel("Spanning (V)")
ax.legend()
fig.tight_layout()
plt.show()
```

Residuen en gereduceerde chi-kwadraat

Residuen zijn `meting - model`. Bekijk of ze een systematisch patroon hebben. Voor de interpretatie van chi-kwadraat nemen we onafhankelijke, normaal verdeelde meetfouten met bekende standaardonzekerheden aan. Er moeten meer meetpunten dan vrije fitparameters zijn.

Python

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

def lineair(t, a, b):
    return a * t + b

tijd = np.array([1, 2, 3, 4, 5], dtype=float)
positie = np.array([1.1, 1.9, 3.2, 3.9, 5.1])
sigma = np.full(5, 0.1)
popt, pcov = curve_fit(lineair, tijd, positie,
                       sigma=sigma, absolute_sigma=True)
residuen = positie - lineair(tijd, *popt)
dof = len(tijd) - len(popt)
chi2 = np.sum((residuen / sigma)**2)
print(f"chi2 = {chi2:.2f}; vrijheidsgraden = {dof}")
print(f"chi2_red = {chi2 / dof:.2f}")

fig, ax = plt.subplots()
ax.errorbar(tijd, residuen, yerr=sigma, fmt="o", capsize=3)
ax.axhline(0, color="black", linewidth=1)
ax.set_xlabel("Tijd (s)")
ax.set_ylabel("Residu (m)")
fig.tight_layout()
plt.show()
```

Hier is `chi2_red = 2.40`. Een waarde rond 1 is een verwachting onder de aannames, geen automatische goedkeuring van het model. Bij weinig vrijheidsgraden zijn grote schommelingen mogelijk. De foutbalken tonen hier de oorspronkelijke meetonzekerheden; gefitte residuen zijn onderling gecorreleerd.

Gemiddelde en standaardfout

Voor onafhankelijke herhalingen onder dezelfde omstandigheden, met minimaal twee metingen. De standaardfout omvat geen systematische onzekerheid, zoals een kalibratiefout.

Python

```python
import numpy as np

metingen = np.array([4.9, 5.1, 5.0, 5.2, 4.8])  # in volt
gemiddelde = np.mean(metingen)
spreiding = np.std(metingen, ddof=1)
standaardfout = spreiding / np.sqrt(len(metingen))
print(f"Standaardafwijking = {spreiding:.3f} V")
print(f"Gemiddelde = {gemiddelde:.3f} +/- {standaardfout:.3f} V")
```

CSV-data inlezen en plotten

Dit voorbeeld gebruikt CSV-tekst in het geheugen en heeft dus geen extern bestand nodig. Voor je eigen bestand vervang je `StringIO(csv_tekst)` door `"metingen.csv"`. Dat pad is relatief aan de huidige werkmap.

Python

```python
from io import StringIO
import pandas as pd
import matplotlib.pyplot as plt

csv_tekst = """tijd;spanning
0;0,12
1;1,05
2;2,08
"""
data = pd.read_csv(StringIO(csv_tekst), sep=";", decimal=",")
print(data)

fig, ax = plt.subplots()
ax.plot(data["tijd"], data["spanning"], "o")
ax.set_xlabel("Tijd (s)")
ax.set_ylabel("Spanning (V)")
fig.tight_layout()
plt.show()
```

Documentatie

- Matplotlib: plottypen [2]
- SciPy: curve_fit en parameteronzekerheden [3]
- NumPy: standaardafwijking [4]
- Pandas: read_csv [5]

Documentatie voor op papier

De nummers bij de links verwijzen naar onderstaande webadressen.

1. https://hva-tn.github.io/Website-Practicum/periode%201/python/
2. https://matplotlib.org/stable/plot_types/index.html
3. https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.curve_fit.html
4. https://numpy.org/doc/stable/reference/generated/numpy.std.html
5. https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html